The user module is:

```
import { Module } from '@nestjs/common';
import { UsersController } from './users.controller';
import { UsersService } from './users.service';
import { TypeOrmModule } from '@nestjs/typeorm';
import { User } from './user.entity';
import { UserSubscriber } from './users.subscriber';
import { Profile } from 'src/profile/profile.entity';
// import { UserSchema } from './user.schema';

@Module({
  imports: [TypeOrmModule.forFeature([User, Profile])],
  controllers: [UsersController],
  providers: [UsersService, UserSubscriber],
  exports: [UsersService],
})
export class UsersModule {}
```

The create user dto is:

```
import {
  IsEmail,
  IsString,
  IsOptional,
  IsBoolean,
  MinLength,
  MaxLength,
  ValidateNested,
} from 'class-validator';
import { Type } from 'class-transformer';
import { CreateProfileDto } from '../../profile/dto/create-profile.dto';

export class CreateUserDto {
  @IsEmail()
  @MaxLength(255)
  email: string;

  @IsString()
  @MinLength(2)
  @MaxLength(50)
  firstName: string;

  @IsString()
```

```
  @MinLength(2)
  @MaxLength(50)
  lastName: string;

  @IsOptional()
  @IsBoolean()
  isActive?: boolean;

  @IsOptional()
  @ValidateNested()
  @Type(() => CreateProfileDto)
  profile?: CreateProfileDto;
}
```

The update user dto:

```
import {
  IsEmail,
  IsString,
  IsOptional,
  MinLength,
  MaxLength,
} from 'class-validator';
import { PartialType } from '@nestjs/mapped-types';
import { CreateUserDto } from './create-user.dto';
export class UpdateUserDto extends PartialType(CreateUserDto) {
  @IsOptional()
  @IsEmail()
  @MaxLength(255)
  email?: string;
  @IsOptional()
  @IsString()
  @MinLength(2)
  @MaxLength(50)
  firstName?: string;
  @IsOptional()
  @IsString()
  @MinLength(2)
  @MaxLength(50)
  lastName?: string;
}
```

The user response is:

```
import { Exclude, Expose, Transform, Type } from 'class-transformer';
import { ProfileResponseDto } from '../../profile/dto/profile-response.dto';
import { PostResponseDto } from '../../post/dto/post-response.dto';

export class UserResponseDto {
  @Expose()
  id: string;

  @Expose()
  email: string;

  @Expose()
  firstName: string;

  @Expose()
  lastName: string;

  @Expose()
  isActive: boolean;

  @Expose()
  @Transform(({ obj }) => `${obj.firstName} ${obj.lastName}`)
  fullName: string;

  @Expose()
  @Type(() => ProfileResponseDto)
  profile?: ProfileResponseDto;

  @Expose()
  @Type(() => PostResponseDto)
  posts?: PostResponseDto[];

  @Expose()
  createdAt: Date;

  @Expose()
  updatedAt: Date;

  @Exclude()
  password?: string;
}
```

The case of handling many-to-many relation:

```
async create(createPostDto: CreatePostDto): Promise<PostResponseDto> {
    const author = await this.userRepository.findOne({
      where: { id: createPostDto.authorId },
    });

    if (!author) {
      throw new NotFoundException(`Author with ID ${createPostDto.authorId} not
found`);
    }

    return await this.dataSource.transaction(async (transactionalEntityManager)
=> {
      const post = this.postRepository.create({
        ...createPostDto,
        author,
      });

      // Handle categories
      if (createPostDto.categoryIds && createPostDto.categoryIds.length > 0) {
        const categories = await transactionalEntityManager.find(Category, {
          where: { id: In(createPostDto.categoryIds) },
        });

        if (categories.length !== createPostDto.categoryIds.length) {
          throw new BadRequestException('Some categories were not found');
        }

        post.categories = categories;
      }

      const savedPost = await transactionalEntityManager.save(post);
      // Load with all relations
      const postWithRelations = await transactionalEntityManager.findOne(Post, {
        where: { id: savedPost.id },
        relations: ['author', 'author.profile', 'categories'],
      });

      return plainToInstance(PostResponseDto, postWithRelations, {
        excludeExtraneousValues: true,
      });
    });
  }
```

To handle one-to-many and many-to-many relation (the same way we handle one-to-one relation when we use @JoinColumn):

```
import {
  Entity,
  PrimaryGeneratedColumn,
  Column,
  ManyToOne,
  ManyToMany,
  JoinTable,
  CreateDateColumn,
  UpdateDateColumn,
  JoinColumn,
} from 'typeorm';
import { User } from '../users/user.entity';
import { Category } from '../category/category.entity';

@Entity('posts')
export class Post {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column()
  title: string;

  @Column('text')
  content: string;

  @Column({ default: false })
  isPublished: boolean;

  // Many-to-One with User
  @ManyToOne(() => User, (user) => user.posts, {
    onDelete: 'CASCADE',
  })
  @JoinColumn({ name: 'authorId' })
  author: User;

  @Column()
  authorId: string;

  // Many-to-Many with Category
  @ManyToMany(() => Category, (category) => category.posts, {
    cascade: true,
  })
```

```
  @JoinTable({
    name: 'post_categories',
    joinColumn: {
      name: 'postId',
      referencedColumnName: 'id',
    },
    inverseJoinColumn: {
      name: 'categoryId',
      referencedColumnName: 'id',
    },
  })
  categories: Category[];

  @CreateDateColumn()
  createdAt: Date;

  @UpdateDateColumn()
  updatedAt: Date;
}
```

****

To upload file using nestjs:

1. Generate resource without crud: *nest g resource file-upload*

---

2. Then we change the module to *import MulterModule*:

```
import { Module } from '@nestjs/common';
import { FileUploadService } from './file-upload.service';
import { FileUploadController } from './file-upload.controller';
import { MulterModule } from '@nestjs/platform-express';
import { diskStorage } from 'multer';

@Module({
  imports: [
    MulterModule.register({
      fileFilter: (req, file, cb) => {
        const t1 = ['image/png', 'image/jpeg', 'image/jpg', 'image/gif']
        if(t1.includes(file.mimetype)) {
          const maxSize = 2 * 1024 * 1024;
          if(file.size > maxSize) {
            console.log("The size is > 2MB")
            cb(null, false)
          } else {
            cb(null, true)
          }
```

```
        } else {
          console.log("Not in type")
          cb(null, false)
        }
      },
      storage: diskStorage({
        destination: './uploads',
        filename: (req, file, cb) => {
          const filename = `${Date.now()}-${file.originalname}`
          cb(null, filename)
        },
      }),
    }),
  ],
  controllers: [FileUploadController],
  providers: [FileUploadService],
})
export class FileUploadModule {}
```

---

3. And the service is:

```
import { Injectable } from '@nestjs/common';
@Injectable()
export class FileUploadService {
  handleFileUpload(file: Express.Multer.File) {
    return { message: 'File uploaded successfully', filePath: file.path };
  }
}
```

---

4. And the controller:

```
import { Controller, Post, UploadedFile, UploadedFiles, UseInterceptors } from
'@nestjs/common';
import { FileUploadService } from './file-upload.service';
import { FileFieldsInterceptor, FileInterceptor } from '@nestjs/platform-
express';

@Controller('file-upload')
export class FileUploadController {
  constructor(private readonly fileUploadService: FileUploadService) {}

  @Post('/upload')
  @UseInterceptors(FileInterceptor('file'))
  uploadFile(@UploadedFile() file: Express.Multer.File) {
    return this.fileUploadService.handleFileUpload(file)
```

```
  }

  @Post('/upload-multiple')
  @UseInterceptors(
    FileFieldsInterceptor([
      { name: 'main', maxCount: 1},
      { name: 'avatar', maxCount: 1}
    ])
  )
  uploadMultipleFiles(@UploadedFiles() files: {
    main?: Express.Multer.File[], avatar?: Express.Multer.File[]
  }){
    return {
      message: 'Files uploaded successfully!',
      avatar: files.avatar?.[0].path,
      background: files.main?.[0].path,
    };
  }
}
```

---

5. And to serve the static files and images from nestjs server, we use ServeStaticModule:

```
import { Module } from '@nestjs/common';
import { AppController } from './app.controller';
import { AppService } from './app.service';
import { ConfigModule } from '@nestjs/config';
import { ServeStaticModule } from '@nestjs/serve-static';
import { join } from 'path';
import { FileUploadModule } from './file-upload/file-upload.module';

@Module({
  imports: [
    ConfigModule.forRoot({
      isGlobal: true
    },
  ),
  ServeStaticModule.forRoot({
      rootPath: join(__dirname, '..', 'uploads'),
      serveRoot: '/uploads',
  }), FileUploadModule
],
  controllers: [AppController],
  providers: [AppService],

})
```

```
export class AppModule { }
```

---

6. And the required packages are:

```
"@nestjs/platform-express": "^11.0.1",
"multer": "^2.0.2",
"@types/multer": "^2.0.0", // for dev
```

****

Note: The specific Redis store package might vary. cache-manager-redis-store is a common choice, but alternatives like cache-manager-ioredis-yet also exist depending on compatibility with the cache-manager version.